

Modélisation pour la performance



Samuel Rohaut

Normalisation vs dénormalisation

Normalisation: garantit l'intégrité, évite les anomalies

Dénormalisation: réduit les jointures, accélère les lectures

Approche hybride souvent optimale pour les applications réelles

Dénormaliser les chemins critiques, garder le reste normalisé

Exemple

Cas d'utilisation: Catalogue e-commerce avec affichage fréquent de produits

products

- product_id (PK)
- product_name
- description
- price
- category_id (FK)

categories

- category_id (PK)
- category_name
- parent_category_id (FK)

product_ratings

- rating_id (PK)
- product_id (FK)
- user_id (FK)
- rating_value

product_display

- product_id (PK)
- product_name
- description
- price
- category_id (FK)
- category_name ← dénormalisé
- parent_category_name ← dénormalisé
- avg_rating ← dénormalisé
- rating_count ← dénormalisé
- main_image_url ← dénormalisé

categories

← gardée pour
administration

product_ratings_detail

← gardée pour
détails rarement
consultés

Choix optimaux des types de données

Impact direct sur stockage, mémoire et performances

Privilégier les types spécifiques vs génériques (`smallint` vs `integer`)

Dimensionner au plus juste (pas de `VARCHAR(1000)` pour des codes courts)

Contraindre les domaines (`CHECK`) pour garantir la qualité

Exemple

Cas: Système de réservation avec millions de transactions quotidiennes

Exemple:

```
status CHAR(1) CHECK (status IN ('A','I','P'))
```

```
VARCHAR(20)
```

Stratégies de partitionnement

Division logique en sous-tables physiques

Méthodes: RANGE (dates), LIST (catégories), HASH (distribution)

Clé idéale = colonne fréquemment filtrée

Facilite maintenance et améliore performances des requêtes ciblées

Exemple

Cas: Système bancaire avec requêtes ciblant des périodes spécifiques

Exemple: `PARTITION BY RANGE (transaction_date)` avec partitions mensuelles

Modélisation pour la concurrence

Identifier et isoler les "hotspots" d'écriture

Éviter les verrous sur tables entières

Utiliser sharding pour compteurs/agrégats fréquemment mis à jour

Privilégier UPDATE ciblé vs UPDATE avec sous-requêtes

Exemple

Cas: Application d'actualités à fort trafic avec compteurs de vues

Exemple: Table `page_view_counters` avec sharding par `page_id % 10`

Schémas dimensionnels

Table de faits centrale (métriques) + tables dimensions (contexte)

Optimisé pour requêtes analytiques et agrégations

Facilite navigation dans hiérarchies (dimensions)

Simplifie requêtes complexes multi-critères

Exemple

Cas: Entrepôt de données pour analyse multidimensionnelle des ventes

Exemple: Table `sales_facts` liée aux dimensions `time_dim`,
`product_dim`, `store_dim`

Clés et identifiants optimisés

Séquences: compactes, rapides, locales

UUIDs: distribution facile, plus larges, index moins efficaces

Impact sur clustering physique et jointures

Choix selon nature du système (centralisé vs distribué)

Exemple

Cas: Système distribué nécessitant génération d'IDs sans coordination centrale

Exemple:

```
user_id BIGSERIAL PRIMARY KEY
```

vs

```
user_id UUID DEFAULT gen_random_uuid()
```

Modèles pour données temporelles

Considérer dimension temporelle dès la conception

Options: tables d'historique, colonnes de validité, partitionnement temporel

Indexer efficacement pour requêtes temporelles (tsrange)

Équilibrer précision temporelle et performances

Exemple

Cas: Système d'assurance avec historique des polices et états à date

Exemple: Colonnes `valid_from`, `valid_to` avec index sur
`tsrange(valid_from, valid_to)`

Contraintes et intégrité référentielle

Garantissent qualité mais impact sur performances d'écriture

Options: immédiat vs différé, partiel vs complet

Alternative: validation applicative pour certains cas

Indexer automatiquement les clés étrangères

Exemple

Cas: Import massif de données hiérarchiques nécessitant validation différée

Exemple:

```
FOREIGN KEY (dept_id)
  REFERENCES departments
  DEFERRABLE INITIALLY DEFERRED
```

Conception orientée requêtes

Adapter structure aux patterns d'accès réels

Privilégier performance des requêtes critiques/fréquentes

Parfois nécessaire de dupliquer/transformer données

Équilibrer besoins lecture/écriture selon ratio d'utilisation

Exemple

Cas: Moteur de recherche e-commerce nécessitant résultats instantanés

Exemple: Table `product_search` avec colonnes combinées et pré-formatées

Techniques avancées

Héritage de tables: alternative au partitionnement

Types composites: structures complexes avec typage fort

JSON/JSONB: flexibilité pour attributs variables/évolutifs

Modèles hybrides: relationnel + document selon besoins